Mathematical Transformations

Module 30 Study Notes • Function Transformer

1. The Goal: Normal Distribution

In feature engineering, a core objective is to ensure that your numerical data is **Normally Distributed** (the classic Gaussian "bell curve").

Why is this important?

- **Linear Models Love Normalcy:** Algorithms grounded in statistics, such as Linear Regression and Logistic Regression, inherently assume that the input features are normally distributed. When this assumption is met, they perform significantly better and converge faster.
- **Tree Models Don't Care:** Algorithms like Decision Trees or Random Forests do not make assumptions about the data's distribution. Mathematical transformations will generally have *zero impact* on their performance.

2. How to Diagnose Your Distribution

Before applying any transformation, you must analyze the current shape of your data. There are three primary ways to do this:

1. **PDF (Probability Density Function):** Using `sns.histplot(kde=True)` to visually inspect the bell curve.
2. **Skewness Metric:** Using Pandas `df['column'].skew()`. A value close to 0 is normal, positive means right-skewed, and negative means left-skewed.
3. **QQ Plot (Quantile-Quantile Plot):** The most reliable method. If the data points fall exactly on the 45-degree diagonal line, the data is perfectly normal. If the points deviate (curve away from the line), the data is skewed.

3. Types of Mathematical Transformations

If your data is skewed, you can apply a mathematical function to every value in the column to force it into a normal distribution.

A. Log Transform (`np.log1p`)

This is the most heavily used transformation. It is highly effective for dealing with **Right-Skewed data** (data with extreme high-value outliers, like income or housing prices). It pulls large values down closer to the mean, compressing the scale.

> **Pro Tip:** Always use `np.log1p(x)` instead of `np.log(x)` . `log1p` actually calculates `log(x + 1)` . This prevents the code from crashing if your dataset contains zeroes, because the log of 0 is mathematically undefined.

B. Square Transform (`x²`)

If your data is **Left-Skewed** (long tail on the negative side), squaring the values can help stretch the distribution back toward normality.

C. Reciprocal (`1/x`) & Square Root (`np.sqrt`)

These are alternative transformations that alter the scale of the data in different ways. Machine learning requires an experimental mindset: you often need to test these alternatives and check which one produces the best QQ Plot and highest model accuracy.

4. Implementation: The FunctionTransformer

Scikit-Learn provides the `FunctionTransformer` class to seamlessly integrate these mathematical formulas into your ML Pipelines.

```
import numpy as np
from sklearn.preprocessing import FunctionTransformer
from sklearn.compose import ColumnTransformer

# 1. Initialize the FunctionTransformer with the desired NumPy function
log_transformer = FunctionTransformer(func=np.log1p)

# 2. Integrate it into a ColumnTransformer
# (Applying Log Transform only to the 'Fare' column)
ct = ColumnTransformer(
    transformers=[
        ('log_fare', log_transformer, ['Fare'])
    ],
    remainder='passthrough'
```

```
)

# 3. Fit and transform your training data
X_train_transformed = ct.fit_transform(X_train)
```

5. Key Takeaways

- Mathematical transformations are used primarily to boost the accuracy of linear algorithms (Linear Regression, Logistic Regression).
- Always rely on the **QQ Plot** to visualize whether a transformation actually helped normalize the data or made it worse.
- The `FunctionTransformer` allows you to wrap any custom Python function (or NumPy function) so that it can be cleanly inserted into a Scikit-Learn Pipeline.